



Relatório do Projeto Computacional de Álgebra Linear Numérica

Afonso Barão (ist1113737)
Afonso Esteves (ist1114004)
Maria Santos (ist1113424)

7 de junho de 2026

Resumo

Encontra-se neste documento o relatório com a resolução das questões do projeto computacional de Álgebra Linear Numérica.

Conteúdo

1	Exercício 1	2
1.1	(a)	2
1.2	(b)	2
1.3	(c)	3
1.4	(d)	6
1.5	(e)	8
2	Exercício 2	9
3	Pergunta 3	10
4	Pergunta 4	12
4.1	(a)	12
4.2	(b)	13
A	Código da Pergunta 2	14
B	Código da Pergunta 4	14

1 Exercício 1

1.1 (a)

Pretende-se mostrar que, se $\rho(I - AX_0) < 1$, então a sucessão abaixo converge quadraticamente para A^{-1} :

$$X_{k+1} = 2X_k - X_kAX_k, \quad k = 0, 1, 2, \dots \quad (1)$$

Para isso, comece-se por notar que para $R_k = I - AX_k$:

$$R_k^2 = I^2 - 2IAX_k + AX_kAX_k = I - A(2X_k - X_kAX_k) = I - AX_{k+1} = R_{k+1}.$$

Observe-se ainda que $R_n \rightarrow 0 \implies AX_n \rightarrow I \implies X_n \rightarrow A^{-1}$. Logo, desde que $R_n \rightarrow 0$ temos que $X_n \rightarrow A^{-1}$. Queremos então mostrar que $\rho(I - AX_0) < 1 \implies R_n \rightarrow 0$:

$$\rho(I - AX_0) < 1 \implies \lim_{n \rightarrow \infty} (I - AX_0)^n = 0 \implies (R_0)^n \rightarrow 0, \quad (2)$$

onde se usou o facto de que $\lim_{n \rightarrow \infty} A^n = 0$ sse $\rho(A) < 1$ (resultado visto em Matemática Computacional). Uma vez que $R_{k+1} = R_k^2$ então $R_k = R_0^{2^k}$. Conclui-se então que, uma vez que $(R_0)^n \rightarrow 0$, então também $R_k = (R_0)^{2^k} \rightarrow 0$.

Para estabelecer a ordem de convergência do método, seja $E_k = A^{-1} - X_k$, queremos mostrar que $\exists M > 0 : \|E_{k+1}\| \leq M\|E_k\|^2$.

Comece-se por notar na igualdade $E_k = A^{-1}R_k$, de onde sai

$$E_{k+1} = A^{-1}R_{k+1} = A^{-1}R_kR_k = A^{-1}R_kAA^{-1}R_k = E_kAE_k \quad (3)$$

Logo $\|E_{k+1}\| = \|E_kAE_k\| \leq \|A\|\|E_k\|^2 \implies X_k \rightarrow A^{-1}$ quadraticamente.

1.2 (b)

Queremos mostrar que:

$$\rho\left(I - \frac{AA^T}{\|A\|_F^2}\right) < 1 \quad (4)$$

Começaremos por provar

1. $\|A\|_F = \sqrt{\sum_{i,j=1}^{n,n} |a_{ij}|^2} = \sqrt{\text{tr}(AA^T)}$;

2. AA^T é definida positiva.

Para provar 1, note-se que

$$\begin{aligned}
(AA^T)_{ij} &= \sum_{k=1}^n a_{ik}a_{kj}^T = \sum_{k=1}^n a_{ik}a_{jk}, \\
(AA^T)_{ii} &= \sum_{k=1}^n |a_{ik}|^2, \\
\text{tr}(AA^T) &= \sum_{i=1}^n (AA^T)_{ii} = \sum_{i=1}^n \sum_{k=1}^n |a_{ik}|^2
\end{aligned}$$

Para provar 2, basta notar no seguinte:

$$x^T AA^T x = (A^T x)^T \cdot (A^T x) = \|A^T x\|_2^2 \geq 0$$

Logo, AA^T é pelo menos semidefinida positiva. Para provar que é de facto definida positiva, basta notar que por hipótese A é não singular, o que faz com que A^T seja não singular. Ou seja, $\|A^T x\| = 0$ sse $x = 0$.

Seja então μ_i um dos valores próprios de $\frac{AA^T}{\|A\|_F^2}$ e γ_i um dos valores próprios de AA^T . Uma vez que AA^T é definida positiva então $\gamma_i > 0$. A partir da propriedade (1) acima temos que:

$$\mu_i = \frac{\gamma_i}{\sum_i \gamma_i} \implies 0 < \mu_i < 1. \quad (5)$$

Seja λ_i um valor próprio de $I - \frac{AA^T}{\|A\|_F^2}$ e x_i o seu vetor próprio associado. Então $(I - \frac{AA^T}{\|A\|_F^2})x_i = x_i - \mu_i x_i = (1 - \mu_i)x_i \implies \lambda_i = 1 - \mu_i$. Como $0 < \mu_i < 1$, também $0 < \lambda_i < 1$. Logo, uma vez que $\forall i, |\lambda_i| < 1 \implies \rho(I - \frac{AA^T}{\|A\|_F^2}) < 1$.

Logo, pela alínea anterior, usando $X_0 = \frac{A^T}{\|A\|_F^2}$, a sucessão (1) converge quadraticamente para A^{-1} .

1.3 (c)

O nosso objetivo agora é a implementação deste método. A ideia seria construir uma função com $\text{Input}(A, X_0, n_{max}, \epsilon)$, onde A é a matriz que queremos inverter e X_0 seria o nosso palpite inicial. O algoritmo da função consiste num *while loop* que calcula a iteração X_{k+1} à custa de X_k , até que o erro $\|E_k\| = \|A^{-1} - X_k\| < \epsilon$. Encontramos dois problemas com esta abordagem.

Pela prova feita na alínea a) a única condição suficiente que conhecemos para garantir convergência é $\rho(I - AX_0) < 1$, daí a função ter como *input* o numero n_{max} , este que é o numero máximo de termos de X_n que vamos calcular para garantir que se $X_n \not\rightarrow A^{-1}$ o código termina na mesma.

O outro problema com que nos deparamos é o de determinar quando $\|E_k\| < \epsilon$. Tal problema surge porque $\|E_k\| < \epsilon \implies \|R_k\| < \|A\|\epsilon$, mas $\|R_k\| < \epsilon \not\Rightarrow C\|E_k\| < \epsilon$. Considere as seguintes implicações: (Vamos por enquanto assumir que $I - R_k$ é invertível e que $\|R_k\| < 1$. Iremos justificar devidamente mais para a frente)

$$R_k = I - AX_k \implies A^{-1} = X_k(I - R_k)^{-1} \implies E_k = X_k((I - R_k)^{-1} - I) \quad (6)$$

Note agora no seguinte. Seja $X \in \mathbb{R}^{n \times n}$ tal que $\det(I - X) \neq 0$ e $\|X\| < 1 \implies \rho(X) < 1$. Seja,

$$S_n = \sum_{i=0}^n X^i \quad (7)$$

Note que, pela propriedade telescópica:

$$S_n - XS_n = I - X^{n+1} \implies S_n = (I - X)^{-1}(I - X^{n+1}) \quad (8)$$

Logo tomando o limite temos que:

$$(I - X)^{-1} = \lim_{n \rightarrow \infty} S_n = I + X + X^2 + X^3 + \dots \quad (9)$$

Podemos então simplificar a expressão de E_k para:

$$E_k = X_k(-I + I + R_k + R_k^2 + R_k^3 + \dots) = X_k(R_k + R_k^2 + R_k^3 + \dots) = X_k R_k (I - R_k)^{-1} \quad (10)$$

Note que $X_{k+1} - X_k = X_k - X_k A X_k = X_k R_k$, logo também temos a seguinte igualdade:

$$E_k = (X_{k+1} - X_k)(I - R_k)^{-1} \quad (11)$$

Usando estas expressões, lembrando que para quaisquer matrizes $A, B \in \mathbb{R}^{n \times n}$, $\|AB\| \leq \|A\|\|B\|$ e considerando a seguinte desigualdade:

$$\|(I - R_k)^{-1}\| = \left\| \sum_{n=0}^{\infty} (R_k)^n \right\| \leq \sum_{n=0}^{\infty} \|(R_k)\|^n = \frac{1}{1 - \|R_k\|} \quad (12)$$

Podemos então concluir que:

$$\frac{\|X_{k+1} - X_k\|}{1 - \|R_k\|} < \epsilon \implies \|E_k\| < \epsilon \quad (13)$$

$$\frac{\|X_k\|\|R_k\|}{1 - \|R_k\|} < \epsilon \implies \|E_k\| < \epsilon \quad (14)$$

Apesar de termos chegado a expressões simpáticas que podemos usar como critérios de paragem ainda temos alguns problemas para resolver. Tudo o que fizemos até aqui exige três condições:

- (i) $\det(I - R_k) \neq 0$.
- (ii) $\|R_k\| < 1$.
- (iii) As normas usadas têm as propriedades $\|AB\| \leq \|A\|\|B\|$ e $\rho(X) \leq \|X\|$.

O problema (iii) surge porque gostaríamos de poder usar a norma de Frobenius devido a ser computacionalmente pouco custosa. Felizmente as duas propriedades das normas induzidas que usámos: $\|AB\| \leq \|A\|\|B\|$ e $\rho(X) \leq \|X\|$ também se aplicam para a norma de Frobenius. Para provar a primeira desigualdade recorde-se que $AB_{ij} = \sum_k a_{ik}b_{kj}$, de onde sai:

$$|AB_{ij}|^2 = \left| \sum_{k=1}^n a_{ik} b_{kj} \right|^2 \quad (15)$$

Logo pela desigualdade de Cauchy-Schwarz temos que:

$$|AB_{ij}|^2 = \left| \sum_{k=1}^n a_{ik} b_{kj} \right|^2 \leq \left(\sum_{k=1}^n |a_{ik}|^2 \right) \left(\sum_{k=1}^n |b_{kj}|^2 \right) \quad (16)$$

Temos então que:

$$\|AB\|_F^2 = \sum_{i=1}^m \sum_{j=1}^p |AB_{ij}|^2 \leq \sum_{i=1}^m \sum_{j=1}^p \left(\sum_{k=1}^n |a_{ik}|^2 \right) \left(\sum_{l=1}^n |b_{lj}|^2 \right) \quad (17)$$

$$\|AB\|_F^2 \leq \left(\sum_{i=1}^m \sum_{k=1}^n |a_{ik}|^2 \right) \cdot \left(\sum_{j=1}^p \sum_{l=1}^n |b_{lj}|^2 \right) \quad (18)$$

$$\|AB\|_F^2 \leq \|A\|_F^2 \|B\|_F^2 \implies \|AB\|_F \leq \|A\|_F \|B\|_F \quad (19)$$

Já a segunda, usando novamente a desigualdade de Cauchy-Schwarz:

$$|(Ax)_i|^2 = \left| \sum_{j=1}^n a_{ij} x_j \right|^2 \leq \left(\sum_{j=1}^n |a_{ij}|^2 \right) \left(\sum_{j=1}^n |x_j|^2 \right) = \left(\sum_{j=1}^n |a_{ij}|^2 \right) \|x\|_2^2 \quad (20)$$

$$\|Ax\|_2^2 = \sum_{i=1}^n |(Ax)_i|^2 \leq \sum_{i=1}^n \left(\sum_{j=1}^n |a_{ij}|^2 \right) \|x\|_2^2 = \left(\sum_{i=1}^n \sum_{j=1}^n |a_{ij}|^2 \right) \|x\|_2^2 \quad (21)$$

Logo:

$$\|Ax\|_2^2 \leq \|A\|_F^2 \|x\|_2^2 \implies \|Ax\|_2 \leq \|A\|_F \|x\|_2 \quad (22)$$

Tomando então x como sendo um vetor próprio ($\neq 0$) com valor próprio associado a $\lambda = \max_i |\lambda_i|$:

$$\|Ax\|_2 = |\lambda| \|x\|_2 \leq \|A\|_F \|x\|_2 \implies |\lambda| \leq \|A\|_F \implies \rho(A) \leq \|A\|_F \quad (23)$$

Quanto ao problema (ii). Sabemos que se $X_k \rightarrow A^{-1} \implies R_k \rightarrow 0$, logo se o nosso palpite inicial X_0 garantir que $X_k \rightarrow A^{-1}$, sabemos que $\exists N > 0 : k > N \implies \|R_k\| < 1$. Assim, desde que $n_{\max}$ seja suficientemente grande o critério de paragem se tornará matematicamente válido a partir de algum k . Isto acontecerá rapidamente se X_0 estiver sobre as condições da alínea a) devido à convergência ser quadrática.

No caso de (i), note que:

$$\det(I - R_k) = \det(I - I + AX_k) = \det(A) \det(X_k) \quad (24)$$

Sabemos por hipótese que $\det(A) \neq 0$, para mostrar que X_k é invertível usaremos um argumento por indução. Começamos por ter o cuidado de colocar um palpite inicial que seja invertível. O nosso objetivo é determinar a inversa de uma matriz por isso não faria sentido que o nosso

palpite inicial para essa inversa não fosse também invertível. Queremos então mostrar que se $\det(X_k) \neq 0 \implies \det(X_{k+1}) \neq 0$. Note que:

$$X_{k+1} = X_k + X_k R_k = X_k(I + R_k) \quad (25)$$

Por hipótese sabemos que $\det(X_k) \neq 0$. Logo basta mostrar que $\det(I + R_k) \neq 0$. Se X_0 estiver nas condições da alínea a) então $\rho(R_0) < 1$ de onde temos que:

$$\rho(R_k) = \rho(R_0^{2^k}) = \rho(R_0)^{2^k} < 1, \quad \forall k \in \mathbb{Z} \quad (26)$$

Temos também que $\det(I + R_k) = 0 \iff 1 + \mu_i = 0$, para pelo menos um i , onde μ_i é valor próprio de R_k . Como $\rho(R_k) < 1 \implies 1 + \mu_i \neq 0$.

Logo desde que X_0 seja invertível (não faria sentido que não fosse), e desde que estejamos sobre as condições da alínea a) então conseguimos garantir que $(I - R_k)$ é invertível.

Tendo resolvido os nosso três impasses, temos todos os resultados teóricos que precisamos para implementar o método em MATLAB, estando o código do mesmo nos anexos.

1.4 (d)

Embora o enunciado do projeto sugira a utilização dos critérios de paragem $\|X_{k+1} - X_k\| < \epsilon$ e $\|R_k\| < \delta$, os desenvolvimentos teóricos da alínea anterior motivaram a adoção complementar das fórmulas (13) e (14). Adicionalmente, foram implementadas funções auxiliares que computam automaticamente um palpite inicial X_0 como *input*, recorrendo diretamente ao resultado da alínea b). Para a matriz de Vandermonde $V([1, 3, 5, 7, 9])$, obteve-se a seguinte tabela:

Critério \ ϵ	0.1	10^{-3}	10^{-6}	10^{-9}	10^{-12}	10^{-15}
Fórmula (13)	35	36	37	37	37	2000(NC)
Fórmula (14)	35	36	37	37	2000(NC)	2000(NC)
$\ X_{k+1} - X_k\ $	0	0	37	37	37	2000(NC)
$\ R_k\ $	34	35	36	37	42	2000(NC)

Tabela 1: Número de iterações necessárias para a convergência de acordo com o critério de paragem e a tolerância (ϵ)

Antes de prosseguirmos com a análise das restantes matrizes, é crucial interpretar esta tabela, pois ela fornece informações valiosas sobre o comportamento dos critérios de paragem.

Observa-se que, para tolerâncias no intervalo de 0.1 a 10^{-9} , quase todos os critérios apresentam um comportamento semelhante, com a notável exceção de $\|X_{k+1} - X_k\| < \epsilon$ para as tolerâncias 0.1 e 10^{-3} . Como a matriz possui uma grande amplitude de valores ($9^4 - 1 = 6560$), a aproximação inicial $X_0 = \frac{A^T}{\|A\|_F^2}$, definida na alínea b), resulta numa matriz com entradas infinitesimais ($\|X_0\| \ll 1$). Sendo $X_0 \approx 0$, deduz-se que $R_0 \approx I$. Consequentemente, como $\|X_1 - X_0\| \leq \|X_0\| \|R_0\|$, obtém-se $\|X_1 - X_0\| \ll 1$, o que leva o critério a parar prematuramente, apesar de o erro real $\|E_k\|$ não ser inferior a ϵ (falso positivo).

Outra discrepância relevante ocorre na Fórmula (14) para $\epsilon = 10^{-12}$. No sistema de ponto flutuante do MatLab (64 bits), o epsilon da máquina é aproximadamente 2.22×10^{-16} , estabele-

cendo um limite inferior físico para o resíduo, ou seja, $\|R_k\| \gtrsim 2.22 \times 10^{-16}$ (aproximadamente maior). À medida que o método converge ($X_k \rightarrow A^{-1}$), a norma $\|X_k\|$ aproxima-se de $\|A^{-1}\|$. Tratando-se de uma matriz mal condicionada, $\|A^{-1}\|$ assume valores muito elevados. Logo, o termo $\|X_k\|\|R_k\|$ sofre uma amplificação que impede a satisfação da Fórmula (14) sempre que $\epsilon \leq \|A^{-1}\| \cdot 2.22 \times 10^{-16}$, o que acontecerá frequentemente para matrizes mal condicionadas.

O facto de nenhum dos critérios convergir para $\epsilon = 10^{-15}$ ilustra precisamente a estagnação do algoritmo ao atingir o limiar da precisão da máquina.

Por fim, é de destacar a velocidade de convergência do método, graças à convergência quadrática, bastou apenas uma iteração adicional (da 36 para a 37) para ganhar 9 dígitos extra de precisão (passando de 10^{-3} para 10^{-12}).

Face ao exposto, no estudo das matrizes subsequentes, será adotada a Fórmula (13) como critério principal de paragem.

Tamanho \ ϵ	0.1	10^{-3}	10^{-6}	10^{-9}	10^{-12}	10^{-15}
$n = 5$	5	8	9	9	10	10
$n = 10$	7	9	10	11	11	12
$n = 15$	8	10	11	11	12	12
$n = 20$	8	10	11	12	12	13
$n = 25$	9	10	12	12	13	13

Tabela 2: Número de iterações até atingir o critério de paragem para a Matriz A_n

Tamanho \ ϵ	0.1	10^{-3}	10^{-6}	10^{-9}	10^{-12}	10^{-15}
$n = 5$	42	43	43	NC	NC	NC
$n = 10$	NC	NC	NC	NC	NC	NC
$n = 15$	NC	NC	NC	NC	NC	NC
$n = 20$	NC	NC	NC	NC	NC	NC
$n = 25$	NC	NC	NC	NC	NC	NC

Tabela 3: Número de iterações até atingir o critério de paragem para a Matriz de Hilbert

Tamanho \ ϵ	0.1	10^{-3}	10^{-6}	10^{-9}	10^{-12}	10^{-15}
$n = 5$	11	12	13	14	14	NC
$n = 10$	16	17	18	18	19	NC
$n = 15$	18	19	20	21	21	NC
$n = 20$	20	21	22	22	23	NC
$n = 25$	22	23	23	24	24	NC

Tabela 4: Número de iterações até atingir o critério de paragem para a Matriz de Lehmer

Olhando primeiro para as tabelas que nos dão o número de iterações para um dado erro, vemos que a matriz A_n é bem condicionada, tendo o método convergido mesmo para uma precisão quase igual à da máquina e com pouquíssimas iterações.

Por outro lado, a matriz de Hilbert é extremamente mal condicionada, tendo o método convergido apenas para matrizes pequenas e com margens de erro relativamente grandes. Pela alínea a) conseguimos garantir a convergência do nosso método se $\rho(I - AX_0) < 1$, mas para matrizes mal condicionadas os valores próprios $\mu_i = 1 - \frac{\lambda_i}{\sum_i \lambda_i} \approx 1$ porque a componente $\frac{\lambda_i}{\sum_i \lambda_i} \approx 0$, logo o raio espectral não será menor que 1 e o método não converge.

A matriz de Lehmer não se mostrou ser tão bem condicionada quanto a matriz A_n , mas comportou-se muito melhor que a matriz de Hilbert, tendo o método convergido para uma matriz de tamanho 25×25 com uma precisão $\epsilon = 10^{-12}$.

Em baixo encontram-se as matrizes A_n , Hilbert e Lehmer (com $n = 4$) obtidas através do método de Newton-Schulz e da função `inv` do MATLAB.

MATLAB (<code>inv(A)</code>)		Newton-Schulz (A^{-1})	
$\begin{bmatrix} 0.1250 & -0.0416 & 0.0137 & -0.0041 \\ -0.0416 & 0.1248 & -0.0412 & 0.0123 \\ 0.0137 & -0.0412 & 0.1235 & -0.0370 \\ -0.0041 & 0.0123 & -0.0370 & 0.1111 \end{bmatrix}$		$\begin{bmatrix} 0.1250 & -0.0416 & 0.0137 & -0.0041 \\ -0.0416 & 0.1248 & -0.0412 & 0.0123 \\ 0.0137 & -0.0412 & 0.1235 & -0.0370 \\ -0.0041 & 0.0123 & -0.0370 & 0.1111 \end{bmatrix}$	
$10^3 \times \begin{bmatrix} 0.0160 & -0.1200 & 0.2400 & -0.1400 \\ -0.1200 & 1.2000 & -2.7000 & 1.6800 \\ 0.2400 & -2.7000 & 6.4800 & -4.2000 \\ -0.1400 & 1.6800 & -4.2000 & 2.8000 \end{bmatrix}$		$10^3 \times \begin{bmatrix} 0.0160 & -0.1200 & 0.2400 & -0.1400 \\ -0.1200 & 1.2000 & -2.7000 & 1.6800 \\ 0.2400 & -2.7000 & 6.4800 & -4.2000 \\ -0.1400 & 1.6800 & -4.2000 & 2.8000 \end{bmatrix}$	
$\begin{bmatrix} 1.3333 & -0.6667 & 0 & 0 \\ -0.6667 & 2.1333 & -1.2000 & -0.0000 \\ 0 & -1.2000 & 3.0857 & -1.7143 \\ 0 & -0.0000 & -1.7143 & 2.2857 \end{bmatrix}$		$\begin{bmatrix} 1.3333 & -0.6667 & -0.0000 & 0.0000 \\ -0.6667 & 2.1333 & -1.2000 & -0.0000 \\ 0.0000 & -1.2000 & 3.0857 & -1.7143 \\ 0.0000 & -0.0000 & -1.7143 & 2.2857 \end{bmatrix}$	

Tabela 5: Comparação entre as inversas obtidas pela função `inv` do MATLAB (esquerda) e pelo método iterativo de Newton-Schulz (direita) para $n = 4, \epsilon = 10^{-12}$.

No caso da comparação entre a função `inv` do MATLAB e o método de Newton-Schulz, os nossos dados iniciais sugerem que não existe nenhuma diferença significativa. Contudo, no caso da matriz de Hilbert, verificamos que, para $n = 9$, o valor da norma de Frobenius da diferença entre a inversa obtida pelo método e a obtida pela função `inv` assumiu um valor de 2.411761×10^8 . Isto indica que a função `inv`, em princípio, possui mecanismos internos para lidar com matrizes extremamente mal condicionadas que o método de Newton-Schulz não tem.

1.5 (e)

Após termos implementando um código para a fórmula da ordem de convergência computacional obtivemos o seguinte gráfico.

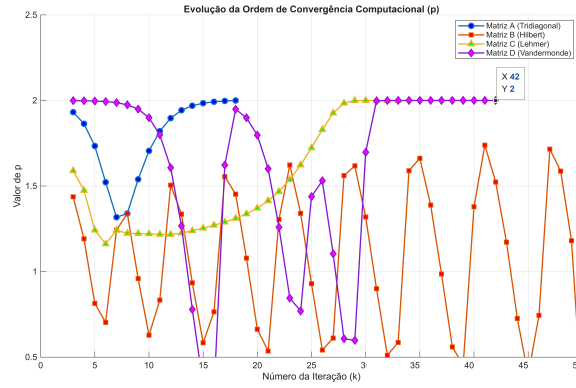


Figura 1: Valor de p com respeito à iteração k

O gráfico ilustra a evolução da ordem de convergência computacional, p , ao longo das iterações para as quatro matrizes distintas. Teoricamente, o método de Newton-Schulz possui convergência $p = 2$. A observação dos dados permite retirar as seguintes conclusões sobre o condicionamento e o comportamento numérico de cada matriz:

- Matriz A_n (Linha Azul): Reforça o facto de ser a matriz mais bem condicionada deste conjunto como já tínhamos visto antes. Após uma ligeira oscilação inicial, o valor sobe rapidamente e estabiliza na perfeição em $p = 2$ por volta da iteração 15. O facto de a linha desaparecer pouco depois da iteração 18 indica que o erro atingiu o limite da precisão da máquina.
- Matriz de Lehmer (Linha Amarela): Apresenta ser bem condicionada mas menos que a matriz A_n . O valor de p sobe de forma gradual e contínua, atingindo a convergência quadrática ($p = 2$) perto da iteração 28. A linha termina logo a seguir, confirmando que o método convergiu com sucesso e esgotou a precisão disponível.
- Matriz de Vandermonde (Linha Rosa): Tem um comportamento estranho. Começa exactamente com $p = 2$, mas sofre uma forte destabilização numérica entre as iterações 10 e 30. No entanto, o método recupera dessa instabilidade, voltando a fixar-se em $p = 2$ (como evidenciado pelo marcador no ponto $X = 42, Y = 2$) até ao final do ciclo. O que indica que apesar de mal condicionada o método converge corretamente para a inversa desta matriz.
- Matriz de Hilbert (Linha Vermelha): Nota-se imediatamente que esta matriz é extremamente mal condicionada. Ao longo das 50 iterações, a linha vermelha oscila de forma caótica, nunca conseguindo estabilizar no valor teórico de 2. Os erros de arredondamento e a instabilidade numérica dominam completamente os cálculos, impedindo que o método de Newton-Schulz convirja de forma eficaz como já tínhamos visto.

2 Exercício 2

Na resolução deste exercício, aplicou-se o método de decomposição QR de Householder apresentado em [1] (pp. 307-314). O objetivo do algoritmo é, a cada passo, refletir o vetor da i -ésima coluna no i -ésimo vetor da base, através de uma reflexão apropriada representada por uma matriz

ortogonal, por forma a que todas as entradas $j > i$ do vetor sejam 0. A função `House` trata de encontrar o vetor u para a reflexão $P = I - 2uu^T$.

Ao compor várias destas reflexões, a transformação resultante é ainda uma matriz ortogonal, a qual vai ser definida por Q . A cada passo, aplicamos este processo à matriz $R(i : , i :)$ até ficarmos com uma matriz triangular superior, a qual será denotada por R .

O código da função `HouseholderDecomposition` encontra-se devidamente comentado em (A).

Era ainda pedido que fosse aplicada a decomposição QR de Householder às matrizes de Lehmer e de Hilbert $L_n, H_n \in \mathbb{R}^{100 \times n}$, $n = 10, \dots, 100$. Foi aplicada, conforme pedido, e os resultados mostraram-se corretos, como esperado. Para confirmação, multiplicaram-se as matrizes QR para cada caso e de facto obteve-se a matriz original, a menos de erros negligenciáveis provenientes do sistema de ponto flutuante.

Pediu-se também que se calculasse as normas de Frobenius de $\|Q_n R_n - L_n\|_F$, $\|Q_n^T Q_n - I\|_F$, $\|Q_n R_n - H_n\|_F$, $\|Q_n^T Q_n - I\|_F$, cujos valores para $n = 10, \dots, 100$ encontram-se na tabela abaixo:

n	$\ Q_{n,L} R_{n,L} - L_n\ _F$	$\ Q_{n,H} R_{n,H} - H_n\ _F$	$\ Q_{n,L}^T Q_{n,L} - I\ _F$	$\ Q_{n,H}^T Q_{n,H} - I\ _F$
10	7.45938739517e-15	1.00629513098e-15	1.38572404594e-14	1.24259303708e-14
20	1.80588085780e-14	1.18426739755e-15	1.89193774316e-14	1.97422755701e-14
30	3.07028816807e-14	1.27313399309e-15	2.11810185655e-14	2.35890801111e-14
40	4.38072253837e-14	1.32550355881e-15	2.31447749599e-14	2.51757661985e-14
50	5.62474198138e-14	1.36148830161e-15	2.35923788113e-14	2.67808944836e-14
60	6.80292229132e-14	1.39274475742e-15	2.50853728519e-14	2.77230222368e-14
70	7.88114127576e-14	1.42022691368e-15	2.59368974489e-14	2.81437039206e-14
80	8.92286904687e-14	1.44500278906e-15	2.59837636204e-14	2.80152625269e-14
90	9.88365946516e-14	1.46262960670e-15	2.61084365727e-14	2.83921366819e-14
100	1.0733402212e-13	1.47772452647e-15	2.63806178067e-14	2.85062660173e-14

Tabela 6: Normas de Frobenius pedidas

Para evitar confusões, denota-se $Q_{n,L}$ pela matriz Q da decomposição QR de Householder da matriz L_n e $Q_{n,H}$ pela matriz Q da decomposição da matriz H_n .

Tendo em conta os resultados obtidos, nomeadamente que os erros da decomposição de Householder são (em geral) da ordem 10^{-14} , é viável intuir que o algoritmo da decomposição é estável. É possível observar que, para cada caso, a decomposição QR é exata e também a matriz Q é ortogonal, a menos de erros de arredondamento devido ao sistema de ponto flutuante.

3 Pergunta 3

Começámos por procurar uma imagem no *Google* e escolhemos uma figura de um *Charmander* disponível tanto a cores quanto a preto e branco. No caso da decomposição SVD da matriz que representa a imagem a preto e branco do *Charmander*, obtivemos um total de 512 valores singulares. As imagens resultantes da redução de qualidade, correspondentes a manter apenas 1%, 10%, 25%, 50% e 75% dos valores, foram:



(a) Imagem Original
Qualidade:100%



(b) 75% ($p = 384$)
Qualidade: 100.00000%



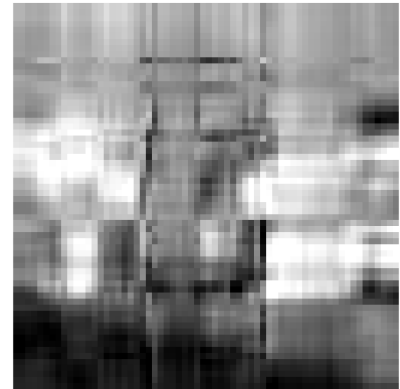
(c) 50% ($p = 256$)
Qualidade: 100.00000%



(d) 25% ($p = 128$)
Qualidade: 100.00000%



(e) 10% ($p = 51$)
Qualidade: 99.99243%



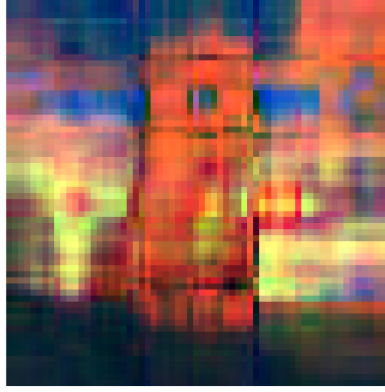
(f) 1% ($p = 5$)
Qualidade: 94.08630%

Figura 2: Resultados da compressão de imagem por SVD no canal de cinzentos, variando a percentagem de valores singulares retidos (p) e respetiva qualidade numérica da reconstrução.

Seguidamente, desenvolvemos um código que replica o mesmo procedimento para cada uma das matrizes dos canais RGB da imagem colorida. Os resultados obtidos ao manter as mesmas percentagens de valores singulares em cada canal foram muito semelhantes aos da versão a preto e branco, visto que não se verifica nenhuma diferença visualmente perceptível em relação à original, exceto no caso em que se preservou apenas 1% dos valores singulares de cada cor. Decidimos também analisar o impacto na imagem ao remover todos os valores singulares de uma determinada cor, sendo os resultados obtidos apresentados em seguida:



(a) Imagem Original
Qualidade 100%
R/G/B:100%



(b) Retenção de 1% ($p = 5$)
R: 90.8480%, G:92.0246%,
B: 94.5754%



(c) Canal Vermelho Removido
R: $p = 0$ (0%)
G/B: $p = 512$ (100%)



(a) Canal Verde Removido
G: $p = 0$ (0%)
R/B: $p = 512$ (100%)



(b) Canal Azul Removido
B: $p = 0$ (0%)
R/G: $p = 512$ (100%)

Figura 4: Decomposição e reconstrução matricial através de SVD nos canais RGB. Subfigura (b): compressão agressiva mantendo apenas 1% dos valores singulares. Subfiguras (c), (d) e (e): isolamento cromático através da anulação total ($p = 0$) de um canal específico em contraste com os restantes a 100% de qualidade.

4 Pergunta 4

4.1 (a)

O objetivo do problema é aproximar o número de condição $\text{cond}_2(A)$ através do método das potências. Para isso, construíram-se duas funções auxiliares `metodo_potencias` e `metodo_potencias_inversas`, respetivamente. Estas funções recebem

- A : a matriz que se quer o número de condição;
- $x^{(0)}$: a iterada inicial com $\|x^{(0)}\|_\infty = 1$, segundo os requerimentos do método;
- ϵ : um critério de paragem.

A implementação da função `metodo_potencias` segue apenas o método descrito no enunciado e não tem qualquer interesse, pelo que o seu código será omitido no relatório.

A implementação da função `metodo_potencias_inversas` é feita da seguinte maneira: uma vez que queremos aplicar o método das potências à matriz A^{-1} , sem a calcular explicitamente, podemos fazer o seguinte:

$$z^{(k)} = A^{-1}x^{(k-1)} \iff Az^{(k)} = x^{(k-1)}. \quad (27)$$

Ou seja, ficamos com um sistema linear por resolver. Para resolver tal sistema, fez-se uso da Decomposição QR de Householder implementada na Pergunta 2 para obter $A = QR$ ($A \in \mathbb{R}^{n \times n}, n \geq n$, pelo que podemos usar tal decomposição). Assim,

$$\begin{aligned} Az^{(k)} = x^{(k-1)} &\iff QRz^{(k)} = x^{(k-1)} \iff \\ Rz^{(k)} = Q^T x^{(k-1)} &\iff z^{(k)} = R^{-1}Q^T x^{(k-1)}. \end{aligned}$$

Os restantes passos da implementação são em tudo semelhantes ao método das potências simples.

Por fim, construiu-se a função `cond2`, que se trata apenas de uma função que divide os outputs de cada uma das funções auxiliares já discutidas e que devolve $\text{cond}_2(A)$. O seu código encontra-se no apêndice (B).

4.2 (b)

Testou-se o programa anterior para as matrizes de Hilbert e Lehmer indicadas no enunciado, tendo-se obtido os seguintes resultados para o número de condição $\text{cond}_2(A)$:

n	H_n
5	4.769787209564975e+05
6	1.495377055537787e+07
7	4.754991156200876e+08
8	1.526349398939327e+10
9	4.934066876329791e+11
10	1.603559364985378e+13

Tabela 7: $\text{cond}_2(A)$ para as matrizes de Hilbert

n	L_n
10	86.4733
100	1.026016544519245e+04
200	4.164695404742911e+04
300	9.524097541280324e+04
400	1.695124496000889e+05
500	2.673177221824455e+05
1000	1.075315524014998e+06

Tabela 8: $\text{cond}_2(A)$ para as matrizes de Lehmer

Tendo estes resultados, é válido intuir que as matrizes de Hilbert são mal condicionadas, pois mesmo para pequenos valores de n (ex: $n = 10$), $\text{cond}_2(A)$ é da ordem 10^{13} . O seu número de condição cresce quase exponencialmente.

Para as matrizes de Lehmer, o crescimento do seu número de condição é mais lento, mas também se torna mal condicionada para valores de $n \approx 1000$.

Referências

- [1] L. Beilina, E. Karchevskii e M. Karchevskii. *Numerical Linear Algebra: Theory and Applications*. Springer, 2017.

A Código da Pergunta 2

```
function [Q,R] = HouseholderDecomposition(A)

% obter a dimensao da matriz A
[rows, cols] = size(A);

% no enunciado diz que a funcao tem de receber uma matriz que nao pode ter
% menos linhas do que colunas
if rows < cols
    error('Erro: O número de colunas é maior que o de linhas.')
end

% guardar as matrizes de iteracao R e Q
R = A;
Q = eye(rows);

% inicializacao da iteracao
for i = 1:min(rows - 1, cols)

    % obtencao do vetor u para a reflexao de householder
    u = House(R(i:rows, i));

    % criacao da nova matriz P
    P_k = eye(rows);
    P_k(i:rows,i:rows) = eye(rows-i+1) - 2.*u*u.';

    % atualizacao da matriz R e Q
    R = P_k * R;
    Q = Q * P_k.';
end

end
```

B Código da Pergunta 4

```

% funcao principal do script, a funcao que calcula o numero de condicao a
% partir das funcoes auxiliares dos metodos das potencias

function [cond2] = cond2(A, x_0, epsilon)

% calcular os dois lambdas atraves das funcoes auxiliares
lambda_max = metodo_potencias(A,x_0,epsilon);
lambda_min = metodo_potencias_inversas(A, x_0,epsilon);

cond2 = lambda_max / lambda_min;

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% em seguida encontra-se a funcao que vai computar min/lambda/

% iremos usar a decomposicao QR de householder ja implementada para
% decompor a matriz A por forma a resolver os sistemas
function [lambda_min] = metodo_potencias_inversas(A, x_0, epsilon)

% inicializacao das variaveis
lambda = [0, Inf];
x = [x_0];
z = [];
n = max(size(x_0));

% decomposicao QR de Householder para resolucao do sistema linear
[Q, R] = HouseholderDecomposition(A);
Rinv = inv(R);

% daqui para a frente a logica e a mesma da funcao anterior
while abs(lambda(end)-lambda(end-1)) >= epsilon
    z = [z, Rinv*Q.'*x(:,end)];
    j = 0;
    for i=1:n
        if x(i,end) ~= 0
            j = i;
            break
        end
    end

    lambda = [lambda, z(j, end)/x(j, end)];
    x = [x, z(:,end)/lambda(end)];

end

% uma vez que o algoritmo das potencias inversas devolve 1/lambda_min, para
% obter lambda_min temos de inverter esse valor
lambda_min = abs(1/lambda(end));

end

```